Object Oriented Programming
Lab Assignment 11

22CS02009
Kumar Snehal

To Run:

1. Please place all the program files in a folder called "Lab_11", this is because they were created under package name Lab_11, not doing so will give error.
2. To run program, go outside the Lab_11 directory and run the following commands:
   a. `javac -cp .:/usr/share/java/mysql-connector-j-9.6.0.jar Lab_11/*.java`
   b. `java -cp .:/usr/share/java/mysql-connector-j-9.6.0.jar Lab_11.EventDrivenSystem`

Part A: Basic Observer Implementation

The system implements a decoupled architecture using the Observer Pattern (check `ObserverPattern.java`.

The core component is the `EventListener` interface which acts as the Observer, declaring the `onEvent(String event)` method.

The `EventSubject` abstract class maintains a thread-safe list (`CopyOnWriteArrayList`) of these listeners and provides the `registerListener()` mechanism.

The `EventSource` class inherits from it and implements `notifyListeners()` , ensuring the event source does not know the specific implementation details of the listeners.

Part B: Multithreading Integration

To ensure concurrent event handling, multithreading is integrated into the notification process. Instead of processing listeners sequentially, the `notifyListeners()` method uses an `ExecutorService` configured with a fixed thread pool (check `EventDriverSystem.java`).

When an event fires, each listener's task is submitted to the executor, meaning each listener runs in a separate thread. This ensures a fast response and non-blocking behavior , meaning listeners do not block each other while executing.

Part C: Multiple Listeners

Three specific event listeners (Observers) are implemented to respond to the events:

- `LoggerListener`: Prints a log message indicating the event was recorded.
- `DisplayListener`: Simulates updating the user interface.
- `AnalyticsListener`: Simulates recording data tracking metrics. Each class processes the notification independently and utilizes `Thread.sleep()` to simulate varying workloads.

Part D: Simulate Events

The `EventDrivenSystem` main class handles the real-world event simulation. It initializes the database and event source, registers all three listeners, and then generates multiple user interactions in quick succession , such as:

- "Button_Clicked"
- "Key_Space_Pressed"
- "Mouse_Double_Clicked"

Part E: Add Database

The `DatabaseManager` class handles all database operations via JDBC (check `DataBaseManager.java`). Upon initialization, it ensures the `Events` table exists with columns for `id`, `name`, and `status`. The workflow operates dynamically:

1. `insertEvent()` stores newly generated events in the database with a default status.
2. A separate background thread waits for all thread futures to resolve via `Future.get()`, then utilizes `updateEventStatus()` to dynamically update the database status to "PROCESSED" once all listeners have successfully finished their execution.
3. Finally, `fetchEventHistory()` is called to retrieve and display the updated records.

Screenshots:

```
kumar-snehal@Kumar-Snehal-PC: ~/OOP                                   kumar-snehal@Kumar-Sneh
kumar-snehal@Kumar-Snehal-PC:~/OOP$ javac -cp .:/usr/share/java/mysql-connector-j-9.6.0.jar Lab_11/*.java
kumar-snehal@Kumar-Snehal-PC:~/OOP$ java -cp .:/usr/share/java/mysql-connector-j-9.6.0.jar Lab_11.EventDrivenSystem

<<< Firing Event >>>: Button_Clicked
[Display] Updating UI for event: 'Button_Clicked' | Thread: pool-1-thread-2
[Logger] Logging event: 'Button_Clicked' | Thread: pool-1-thread-1

<<< Firing Event >>>: Key_Space_Pressed
[Analytics] Tracking data for event: 'Button_Clicked' | Thread: pool-1-thread-3
[Logger] Logging event: 'Key_Space_Pressed' | Thread: pool-1-thread-4
[Display] Updating UI for event: 'Key_Space_Pressed' | Thread: pool-1-thread-5

<<< Firing Event >>>: Mouse_Double_Clicked
[Analytics] Tracking data for event: 'Key_Space_Pressed' | Thread: pool-1-thread-1
[Logger] Logging event: 'Mouse_Double_Clicked' | Thread: pool-1-thread-4
[Display] Updating UI for event: 'Mouse_Double_Clicked' | Thread: pool-1-thread-2
[Analytics] Tracking data for event: 'Mouse_Double_Clicked' | Thread: pool-1-thread-5
```

```
[Database] Updated Event ID 4 to status: PROCESSED

[Database] Updated Event ID 5 to status: PROCESSED

[Database] Updated Event ID 6 to status: PROCESSED

=== Event History from Database ===
ID: 1 | Name: Button_Clicked | Status: PROCESSED
ID: 2 | Name: Key_Space_Pressed | Status: PROCESSED
ID: 3 | Name: Mouse_Double_Clicked | Status: PROCESSED
ID: 4 | Name: Button_Clicked | Status: PROCESSED
ID: 5 | Name: Key_Space_Pressed | Status: PROCESSED
ID: 6 | Name: Mouse_Double_Clicked | Status: PROCESSED
==================================

kumar-snehal@Kumar-Snehal-PC:~/OOP$
```